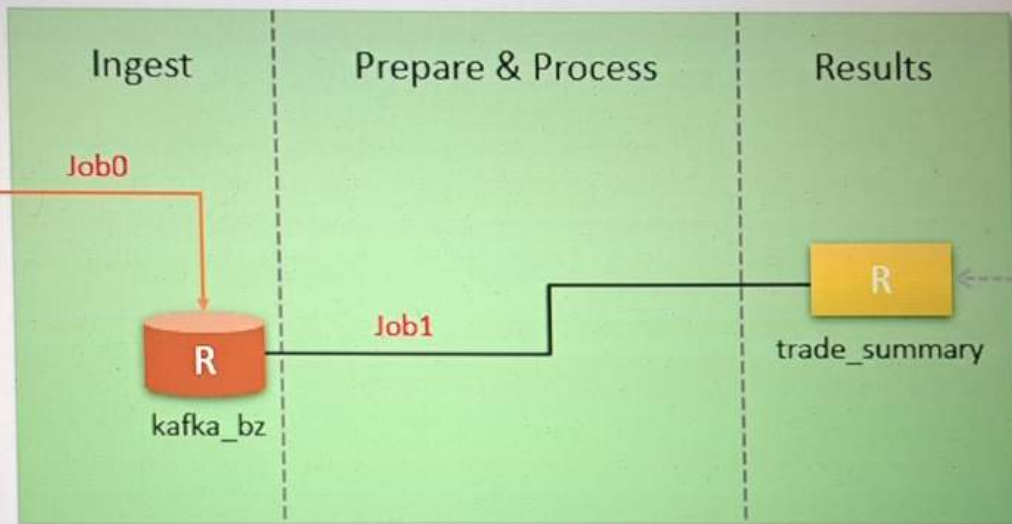


Source System



```
"CreatedTime": "2019-02-05 10:05:00",
"Type": "BUY",
"Amount": 500,
"BrokerCode": "ABX"
```

Data Engineering Platform



Consumer Application



Trade Summary Chart

kafka_bz

key	value
2019-02-05	{"CreatedTime": "2019-02-05 10:48:00", "Type": "SELL", "Amount": 500, "BrokerCode": "ABX"}
2019-02-05	{"CreatedTime": "2019-02-05 10:25:00", "Type": "SELL", "Amount": 400, "BrokerCode": "ABX"}

trade_summary

start	end	TotalBuy	TotalSell
2019-02-05T10:00:00Z	2019-02-05T10:15:00Z	800	0
2019-02-05T10:15:00Z	2019-02-05T10:30:00Z	600	400
2019-02-05T10:30:00Z	2019-02-05T10:45:00Z	900	0
2019-02-05T10:45:00Z	2019-02-05T11:00:00Z	0	500



```
{ "CreatedTime": "2019-02-05 10:05:00",  
  "Type": "BUY",  
  "Amount": 500,  
  "BrokerCode": "ABX" }
```

```
{ "CreatedTime": "2019-02-05 10:12:00",  
  "Type": "BUY",  
  "Amount": 300,  
  "BrokerCode": "ABX" }
```

Input



2019-02-05T10:00:00Z

2019-02-05T10:15:00Z



2019-02-05T10:00:00Z

2019-02-05T10:15:00Z

800

0

```
{  "CreatedTime": "2019-02-05 10:12:00",  
  "Type": "BUY",  
  "Amount": 300,  
  "BrokerCode": "ABX" }
```

```
{  "CreatedTime": "2019-02-05 10:05:00",  
  "Type": "BUY",  
  "Amount": 500,  
  "BrokerCode": "ABX" }
```

2019-02-05T10:00:00Z

2019-02-05T10:15:00Z



State Store

2019-02-05T10:00:00Z

2019-02-05T10:15:00Z

800

0



```
{ "CreatedTime": "2019-02-05 10:20:00",  
  "Type": "BUY",  
  "Amount": 600,  
  "BrokerCode": "ABX" }
```

```
{ "CreatedTime": "2019-02-05 10:40:00",  
  "Type": "BUY",  
  "Amount": 900,  
  "BrokerCode": "ABX" }
```

Input



2019-02-05T10:30:00Z

2019-02-05T10:45:00Z

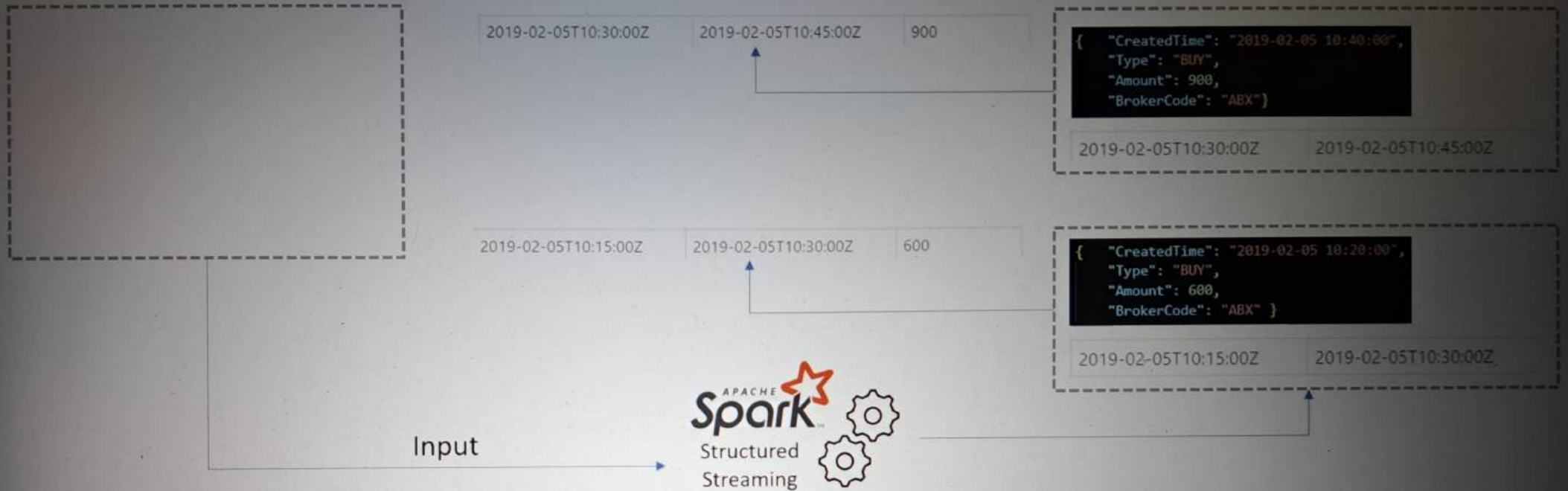
2019-02-05T10:15:00Z

2019-02-05T10:30:00Z

State Store

2019-02-05T10:00:00Z	2019-02-05T10:15:00Z	800	0
----------------------	----------------------	-----	---





State Store

2019-02-05T10:00:00Z	2019-02-05T10:15:00Z	800	0
2019-02-05T10:15:00Z	2019-02-05T10:30:00Z	600	0
2019-02-05T10:30:00Z	2019-02-05T10:45:00Z	900	0




```
{ "CreatedTime": "2019-02-05 10:25:00",  
  "Type": "SELL",  
  "Amount": 400,  
  "BrokerCode": "ABX"}
```

```
{ "CreatedTime": "2019-02-05 10:48:00",  
  "Type": "SELL",  
  "Amount": 500,  
  "BrokerCode": "ABX"}
```

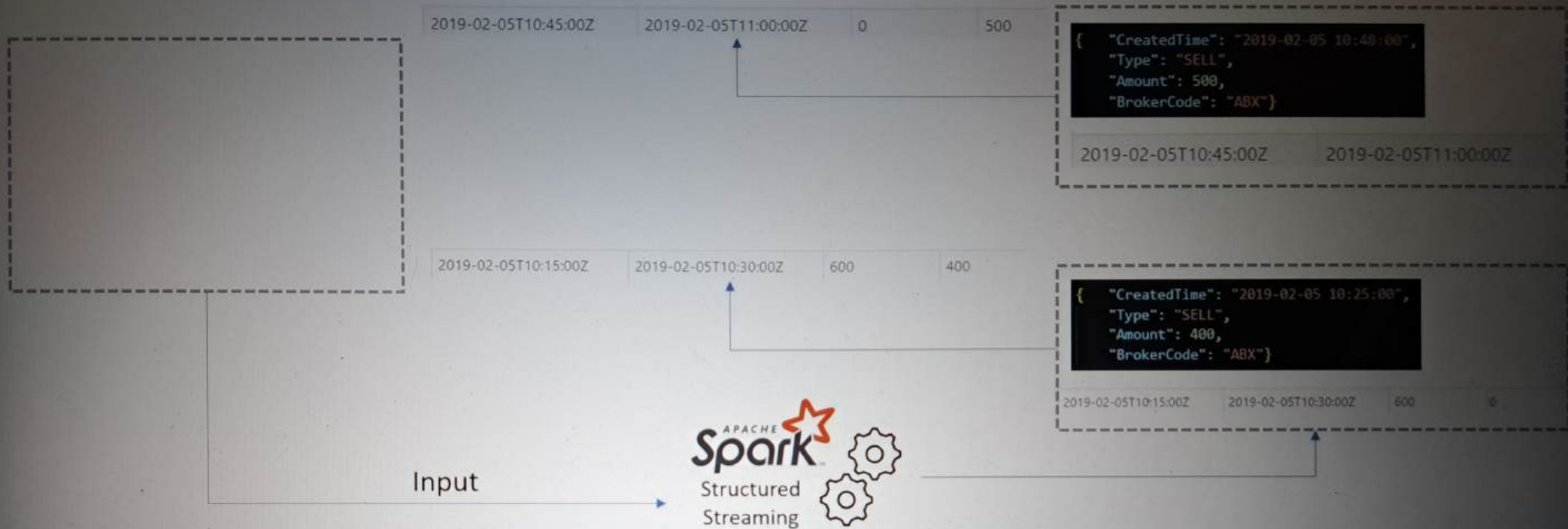
Input



State Store

2019-02-05T10:00:00Z	2019-02-05T10:15:00Z	800	0
2019-02-05T10:15:00Z	2019-02-05T10:30:00Z	600	0
2019-02-05T10:30:00Z	2019-02-05T10:45:00Z	900	0





State Store

2019-02-05T10:00:00Z	2019-02-05T10:15:00Z	800	0
2019-02-05T10:15:00Z	2019-02-05T10:30:00Z	600	400
2019-02-05T10:30:00Z	2019-02-05T10:45:00Z	900	0
2019-02-05T10:45:00Z	2019-02-05T11:00:00Z	0	500





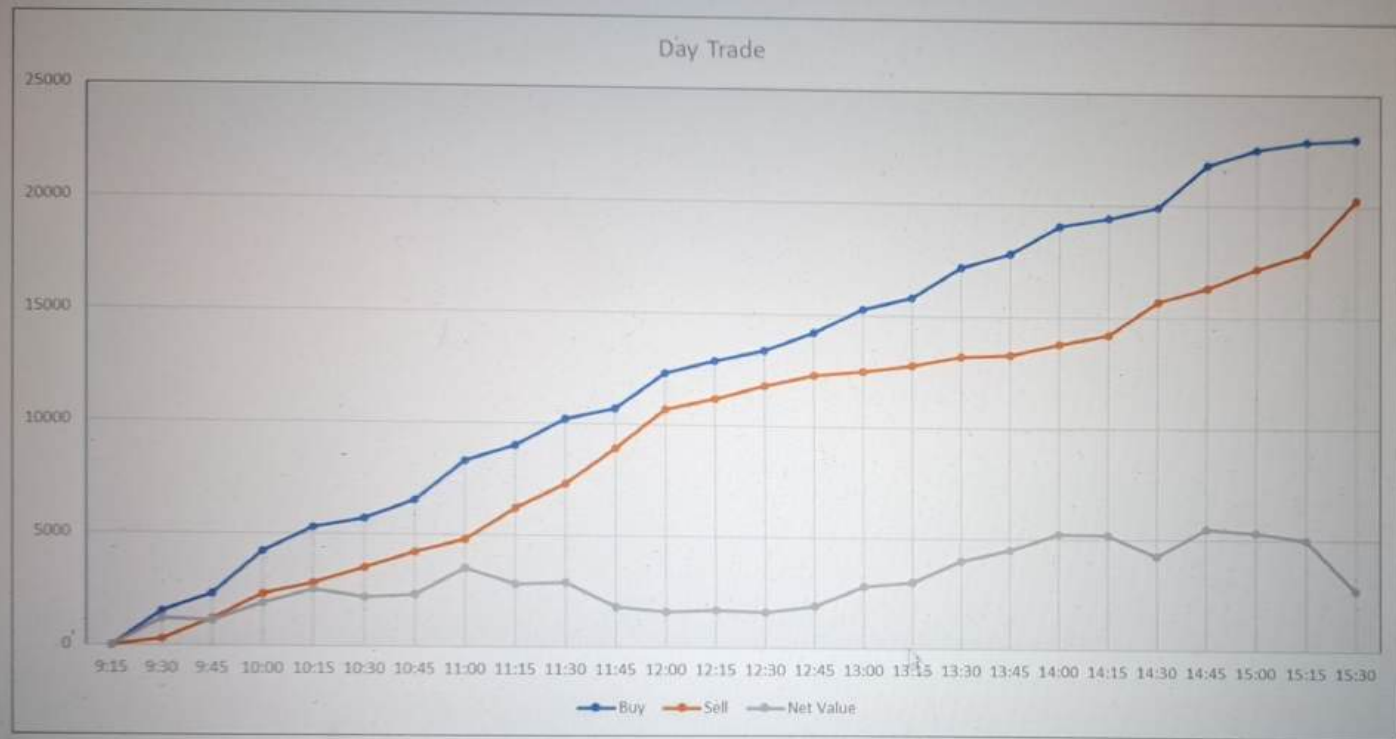
What is a watermark?

How long do you want to wait for a late record?

How to decide your watermark

- What is the maximum possible delay?
- When late records are not relevant?





6 hours

Need 99.99% accuracy – 30 Minutes



22-tumbling-time-window Python ☆

File Edit View Run Help Last edit was 1 minute ago Provide feedback

▶ Run all

● Unknown ▼

Share

Publish

```
.withColumn("CreatedTime", expr("to_timestamp(createdtime, 'yyyy-MM-dd HH:mm:ss')"))  
.withColumn("Buy", expr("case when Type == 'BUY' then Amount else 0 end"))  
.withColumn("Sell", expr("case when Type == 'SELL' then Amount else 0 end"))
```

```
def getAggregate(self, trades_df):  
    from pyspark.sql.functions import window, sum  
    return (trades_df.withWatermark("CreatedTime", "30 minutes")  
            .groupBy(window(trades_df.CreatedTime, "15 minutes"))  
            .agg(sum("Buy").alias("TotalBuy"),  
                 sum("Sell").alias("TotalSell"))  
            .select("window.start", "window.end", "TotalBuy", "TotalSell")  
            )
```

```
def saveResults(self, results_df):  
    print(f"\nStarting Silver Stream...", end='')  
    return (results_df.writeStream  
            .queryName("trade-summary")  
            .option("checkpointLocation", f"{self.base_data_dir}/checkpoint/trade_summary")  
            .outputMode("complete")  
            .toTable("trade_summary")  
            )  
    print("Done")
```

```
def process(self):  
    kafka_df = self.readBronze()  
    trades_df = self.getTrade(kafka_df)  
    results_df = self.getAggregate(trades_df)  
    query = self.saveResults(results_df)
```

